Campus Food Pre-Order and Pickup Management System

Context

A university campus has food outlets that serve students and staff during short breaks, lunch hours, workshops, placement activities and campus events. Each outlet offers a variety of food and beverage items, with limited quantities available during different pickup periods.

Currently, students place orders at the counter or through messages and informal communication. As demand grows, the university faces long queues, orders being accepted without considering item availability, multiple orders competing for limited quantities, limited visibility into order status, delays in confirming and preparing orders, reserved quantities remaining unavailable after cancellations, temporary item unavailability, and different stakeholders requiring different information and capabilities.

Design and build a working prototype of a Campus Food Pre-Order and Pickup Management System. Your team must study the business situation, identify the relevant stakeholders, and determine the workflows, business rules and validations required for the system to operate reliably. The solution should support a meaningful food-order lifecycle, from an initial pre-order request to an appropriate outcome.

Multiple valid approaches are possible. Your solution should be internally consistent, usable and explainable by the team.

Business Conditions

While designing your solution, consider the following minimum business conditions:

1. A pre-order is associated with a customer, one or more menu items, the requested quantity of each item, and a defined pickup slot.
2. Each menu item has a limited quantity available for a pickup slot. Availability must consider quantities committed to submitted or confirmed orders. Once a valid pre-order is successfully submitted, the requested item quantities must be reserved for the selected pickup slot until the order is rejected, cancelled or completed.
3. A menu item must not be committed beyond its available quantity for the selected pickup slot.
4. A pre-order must be prevented from being submitted if the complete requested quantity of any selected item is unavailable for the chosen pickup slot.
5. A menu item may be temporarily unavailable because it is sold out, cannot be prepared, has an ingredient shortage, or is affected by another operational restriction.
6. A successfully submitted pre-order must pass through a confirmation process before it becomes a confirmed order.
7. A rejected pre-order is considered closed. Rejecting a pre-order must release all associated item quantities so that they become available for other valid pre-orders in the same pickup slot. To order different items, quantities or a different pickup slot, the customer must create a new pre-order.
8. An appropriate end user may cancel a confirmed order before its preparation begins. Cancelling the order must release all associated item quantities so that they become available for other valid pre-orders in the same pickup slot.
9. Once preparation has begun, the order cannot be cancelled by the customer.
10. A confirmed order should progress through meaningful stages until it is ready for pickup and collected. The customer should be able to view the current order status.
11. When an order is collected, it should be treated as complete and must not be available for collection again.
12. Partial fulfilment, splitting an order across different pickup slots, and automatic substitutions of unavailable items are not required. If the complete order cannot be fulfilled, the pre-order must not proceed, and the system should communicate the reason clearly.

Your team may introduce additional assumptions, provided they are documented and applied consistently.

Initial Campus Menu Data

Use the following as initial business data. Teams may add reasonable details or additional sample records for testing and demonstration.

Menu Item	Category	Quantity per Pickup Slot	Current Information
Veg Sandwich	Snacks	20	8 units committed for the 13:00–13:30 pickup slot on 4 August 2026 only
Masala Dosa	South Indian	15	All units available
Paneer Rice Bowl	Meals	12	5 units committed for the 12:30–13:00 pickup slot on 4 August 2026 only
Cold Coffee	Beverages	18	Temporarily unavailable for all pickup slots on 4 Aug 2026
Fruit Bowl	Healthy Meals	10	4 units committed for the 13:00–13:30 pickup slot on 4 August 2026 only

Persistence can be in-memory, local files, JSON, CSV, SQLite or another lightweight local approach.

Constraints

- 90 minutes to complete.
- Build a locally runnable prototype with a UI (Streamlit, web-based, or any framework).
- Use Python as the preferred language. C++ or Java are also acceptable.
- No cloud deployment, external APIs, paid services, complex infrastructure, or access to university production systems.
- Authentication is optional. If implemented, explain why it was prioritized.
- Each team must use a shared GitHub repository reflecting meaningful and incremental collaboration. Commit count alone will not measure individual contribution; evaluators may use repository history as supporting evidence of how the team divided, integrated and validated its work.

Deliverables

1. **Working Prototype**: A locally runnable application demonstrating a meaningful food pre-order and pickup lifecycle with connected workflows, consistent handling of pre-orders, item quantities, pickup slots and order states, relevant business validations, and conflict or failure handling. At least one working end-to-end flow must be demonstrated. A smaller, reliable and well-explained solution may score higher than a larger application containing generated code that the team cannot validate or explain.
2. **GitHub Repository**: Must contain source code, run instructions (prerequisites, startup, setup), documented assumptions, scope prioritized, features excluded, important trade-offs, known limitations, and the consulting pitch.
3. **Two-Slide Consulting Pitch:**
    a. Slide 1 — Problem Understanding: Your understanding of the problem, stakeholders/personas identified, workflows prioritized, important business rules discovered, key assumptions, and scope selected for 90 minutes.
    b. Slide 2 — Solution & Engineering Decisions: High-level approach, important design decisions, key trade-offs, testing/validation approach, known limitations, and what the team would improve next.
4. **Final Demonstration Covering**: business problem understanding, stakeholders and workflows identified, the working application, a successful end-to-end scenario, a conflict/invalid/failure scenario, design decisions, testing performed, assumptions and trade-offs, known limitations, and recommended next steps. Evaluators may provide an additional scenario and may direct questions to any team member — each member should be prepared to explain their contribution, one business rule, one part of the solution, one assumption or trade-off, and how their work was validated.

Use of AI Tools

This is an open-book, AI-assisted software engineering challenge. Teams may use GitHub Copilot, Copilot Chat, Claude, GPT-based models, IDE assistance, official documentation, internet search, and AI-generated code, tests, diagrams, documentation and presentation content.

Using AI is not considered cheating. However, the team remains responsible for understanding the problem, identifying requirements, choosing the solution design, reviewing generated output, detecting incorrect or unnecessary suggestions, running and validating generated code, ensuring the implementation matches the business problem, and explaining every submitted artefact.

The objective is not to demonstrate how quickly AI can generate an application. The objective is to demonstrate how effectively the team can direct, question, validate and improve AI-assisted work while retaining ownership of the final solution.

Evaluation Criteria

Criterion	Weight	What We Look For
Problem Understanding	20%	Did the team identify stakeholders, workflows, business rules and scope from the problem statement?
Implementation & Completeness	20%	Does the solution work? How many flows are functional? Is the code well-structured and explainable?
AI Usage	20%	How effectively did the team leverage AI for design, code, iteration and validation?
Collaboration & Communication	20%	How well did the team collaborate? Is the consulting pitch clear and reasoned?
Quiz	20%	Hackathon quiz conducted earlier

Bonus Feature

If your core system is functional and the primary use cases are working, implement one additional capability that enhances the system. Think about what would make this tool genuinely more useful for a campus community — notifications, analytics, calendar integration, smart suggestions, or something else entirely. Creativity counts.

> Note: Bonus is only considered if the core functionality is demonstrably working.

Duration: 90 minutes
Team Size: As announced
Tools: VS Code + GitHub Copilot